

1. Find Middle (Second Middle)

```
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
return slow;
```

1b. Find Middle (First Middle)

```
ListNode slow = head, fast = head;
while (fast.next != null && fast.next.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
return slow;
```

2. Node Before Middle

```
ListNode prev = null, slow = head, fast = head;
while (fast != null && fast.next != null) {
    prev = slow;
    slow = slow.next;
    fast = fast.next.next;
}
return prev;
```

3. Delete Middle

```
ListNode prev = null, slow = head, fast = head;
while (fast != null && fast.next != null) {
    prev = slow;
    slow = slow.next;
    fast = fast.next.next;
}
if (prev != null) prev.next = slow.next;
return head;
```

4. Reverse Linked List

```
ListNode prev = null, curr = head;
while (curr != null) {
    ListNode next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
}
return prev;
```

5. Detect Cycle + Find Start

```
ListNode slow = head, fast = head;
```

```

while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow == fast) break;
}
if (fast == null || fast.next == null) return null;
slow = head;
while (slow != fast) {
    slow = slow.next;
    fast = fast.next;
}
return slow;

```

6. Remove Nth Node From End

```

ListNode dummy = new ListNode(0);
dummy.next = head;
ListNode slow = dummy, fast = dummy;
for (int i = 0; i <= n; i++) fast = fast.next;
while (fast != null) {
    slow = slow.next;
    fast = fast.next;
}
slow.next = slow.next.next;
return dummy.next;

```

7. Merge Two Sorted Lists

```

ListNode dummy = new ListNode(0), curr = dummy;
while (l1 != null && l2 != null) {
    if (l1.val < l2.val) {
        curr.next = l1; l1 = l1.next;
    } else {
        curr.next = l2; l2 = l2.next;
    }
    curr = curr.next;
}
curr.next = (l1 != null) ? l1 : l2;
return dummy.next;

```

8. Intersection of Two Lists

```

ListNode a = headA, b = headB;
while (a != b) {
    a = (a == null) ? headB : a.next;
    b = (b == null) ? headA : b.next;
}
return a;

```

9. Dummy Node Usage

```

ListNode dummy = new ListNode(0);
dummy.next = head;
// simplifies edge cases like deletion at head

```

10. Golden Rules & Patterns

- Slow/Fast: middle, cycle
- Dummy Node: deletion, merging
- Two Pass vs One Pass: optimize with two pointers
- Reverse: 3-pointer pattern
- Intersection: pointer switching trick